

南开大学

汇编语言与逆向技术课程实验报告

实验二：dec2hex



学院：密码与网络空间安全学院

专业：信息安全

学号：2412266

姓名：杨滢

班级：信息安全

1 实验时间

第 7 周星期一 9-10 节

2 实验目的

1. 熟悉汇编语言的数据传送、寻址和算术运算
2. 熟悉汇编语言过程的定义和使用
3. 熟悉十进制和十六进制的数制转换

3 实验环境

MASM32 编译环境、Windows 命令行窗口

4 实验过程

编写汇编程序 dec2hex.asm，编译成 dec2hex.exe。将 Windows 命令行输入的十进制无符号整数，转换成对应的十六进制整数，输出在 Windows 命令行中。

1. StdIn 函数从控制台读取字符串到缓冲区
2. dec2dw 将字符串转十进制整数
3. dw2hex 十进制整数转十六进制字符串
4. StdOut 函数将十六进制数字字符串输出到命令行
5. 使用 ml 将 dec2hex.asm 文件汇编到 dec2hex.obj 目标文件
6. 使用 link 将目标文件 dec2hex.obj 链接成 dec2hex.exe 可执行文件

5 dec2hex.asm 源代码及说明

5.1 源代码

乘法操作版本的 dec2hex:

```
1 .386                ; 指定使用 80386 或更高处理器指令集
2 .model flat, stdcall ; 使用平坦内存模型, stdcall 调用约定
3 option casemap:none ; 区分大小写
4
5 include \masm32\include\masm32.inc ; 包含 StdIn, StdOut 等 I/O 函数
6 include \masm32\include\kernel32.inc ; 包含 Windows API 声明
7 includelib \masm32\lib\masm32.lib ; 链接 masm32.lib (提供 StdIn/StdOut)
8 includelib \masm32\lib\kernel32.lib ; 链接 kernel32.lib (提供 ExitProcess)
```

```
9
10 ; 声明自定义过程的接口
11 dec2dw PROTO :DWORD ; dec2dw 接收一个 DWORD 参数
12 dw2hex PROTO :DWORD, :DWORD ; dw2hex 接收两个参数：数值 和 输出缓冲区地址
13
14 .data
15     ; 提示用户输入的字符串
16     prompt    BYTE "Please input a decimal number (0 ~ 4294967295): ", 0
17               ; 以 0 结尾的 ASCII 字符串
18
19     ; 显示转换结果的前缀
20     resultMsg BYTE "The hexadecimal number is : ", 0
21
22     ; 换行符：回车(CR) + 换行(LF) + 字符串结束
23     newline   BYTE 0Dh, 0Ah, 0 ; 0Dh='\r', 0Ah='\n'
24
25     ; 输入缓冲区
26     inputBuf  BYTE 20 DUP(?) ; 20 个未初始化字节，用于读取输入
27
28     ; 输出缓冲区
29     hexStr    BYTE 9 DUP(?) ; 如 "00000064" + '\0'
30
31 .code
32 dec2dw PROC stringAddr:DWORD
33     push ebx      ; 保存寄存器
34     push ecx
35     push edx
36     push esi
37
38     mov esi, stringAddr ; ESI 指向输入字符串首地址
39     xor eax, eax      ; EAX 清零，用于累加结果
40
41 L1:                ; 循环处理每个字符
42     movzx ebx, byte ptr [esi] ; 读取当前字符
43     cmp bl, 0        ; 判断是否到达字符串末尾
44     je done          ; 是 → 跳出循环
45
46     cmp bl, '0'      ; 检查是否小于 '0'
47     jb invalid       ; 小于则非法
48     cmp bl, '9'      ; 检查是否大于 '9'
49     ja invalid       ; 大于则非法
50
```

```
51     ; 正常数字字符处理: result = result * 10 + (digit - '0')
52     imul eax, eax, 10          ; EAX = EAX * 10
53     sub bl, '0'                ; 将字符 '0'~'9' 转为数字 0~9
54     add eax, ebx               ; 累加当前数字
55     inc esi                    ; 指向下一个字符
56     jmp L1                     ; 继续处理下一位
57
58 invalid:                      ; 遇到非法字符时返回 0
59     mov eax, 0
60
61 done:
62     ; 恢复寄存器值
63     pop esi
64     pop edx
65     pop ecx
66     pop ebx
67     ret                        ; 返回调用者
68 dec2dw ENDP                  ; 过程结束
69
70 dw2hex PROC value:DWORD, buffer:DWORD
71     push eax                  ; 保存所有使用的寄存器
72     push ebx
73     push ecx
74     push edx
75     push esi
76
77     mov esi, buffer           ; ESI 指向输出缓冲区
78     mov ebx, value            ; EBX 存放要转换的数值
79     mov ecx, 8                ; 循环 8 次
80
81 hex_loop:
82     rol ebx, 4                ; 左旋转 4 位, 把最高 4 位移到最低位
83     mov dl, bl                ; 取 BL (低 8 位) 中的低 4 位
84     and dl, 0Fh               ; 屏蔽高 4 位, 只保留低 4 位 (0~15)
85     cmp dl, 9                 ; 判断是否为 0~9
86     jbe is_digit              ; 9 → 是数字, 直接转 ASCII
87     add dl, 7                 ; >9 → 是 A~F, 需加 7
88
89 is_digit:
90     add dl, '0'               ; 转为 ASCII 字符
91     mov [esi], dl             ; 存入输出缓冲区
92     inc esi                   ; 指向下一个位置
```

```
93     loop hex_loop      ; ECX 减 1, 若不为 0 则继续循环
94
95     mov byte ptr [esi], 0 ; 添加字符串结束符 '\0'
96
97     ; 恢复寄存器
98     pop esi
99     pop edx
100    pop ecx
101    pop ebx
102    pop eax
103    ret                ; 返回
104 dw2hex ENDP
105
106 start:
107     ; 输出提示信息
108     invoke StdOut, addr prompt ; 调用 StdOut 显示输入提示
109
110     ; 读取用户输入的字符串
111     invoke StdIn, addr inputBuf, 20
112             ; 从控制台读最多 19 字符 + 自动加 '\0'
113
114     ; 去除输入末尾的换行符 (\n 或 \r\n)
115     invoke lstrlen, addr inputBuf ; 获取字符串长度
116     .if eax > 0                ; 如果长度大于 0
117         dec eax                ; 计算最后一个字符索引
118         lea ebx, inputBuf      ; EBX = 缓冲区起始地址
119         add ebx, eax           ; EBX 指向最后一个字符
120         ; 检查是否为换行符 (10 = \n, 13 = \r)
121         .if byte ptr [ebx] == 10 || byte ptr [ebx] == 13
122             mov byte ptr [ebx], 0 ; 替换为字符串结束符
123         .endif
124     .endif
125
126     ; 调用 dec2dw 将字符串转为整数
127     invoke dec2dw, addr inputBuf ; 参数: 字符串地址
128     mov ebx, eax                ; 保存转换结果到 EBX
129
130     ; 调用 dw2hex 将整数转为十六进制字符串
131     invoke dw2hex, ebx, addr hexStr ; 参数: 数值 和 输出缓冲区
132
133     ; 输出转换结果
134     invoke StdOut, addr resultMsg ; 输出前缀
```

```
135 invoke StdOut, addr hexStr ; 输出十六进制字符串
136 invoke StdOut, addr newline ; 换行
137
138 ; 静默等待用户按键
139 invoke StdIn, addr inputBuf, 2
140             ; 再次调用 StdIn, 等待用户按任意键
141             ; 输入将被忽略, 仅用于暂停程序
142
143 ; 正常退出程序
144 invoke ExitProcess, 0 ; 调用 Windows API 退出程序, 返回码 0
145
146 end start             ; 程序结束
```

移位操作版本的 dec2dw 过程:

```
1 dec2dw PROC stringAddr:DWORD
2     push ebx
3     push ecx
4     push edx
5     push esi
6     mov esi, stringAddr ; ESI -> 字符串首地址
7     xor eax, eax        ; EAX = 0, 用于累加结果
8 L1:
9     movzx ebx, byte ptr [esi] ; 读取当前字符
10    cmp bl, 0            ; 是否到结尾?
11    je done              ; 是 -> 结束
12    cmp bl, '0'          ; 是否 < '0'?
13    jb invalid           ; 是 -> 非法
14    cmp bl, '9'          ; 是否 > '9'?
15    ja invalid           ; 是 -> 非法
16    ; 使用位移实现 ×10
17    mov ecx, eax         ; 保存当前值
18    shl eax, 1           ; EAX = EAX * 2
19    shl ecx, 3           ; ECX = ECX * 8
20    add eax, ecx         ; EAX = EAX * 10
21    sub bl, '0'          ; 字符转数字
22    add eax, ebx         ; 累加
23    inc esi              ; 指向下一个字符
24    jmp L1               ; 继续循环
25
26 invalid:
27     xor eax, eax        ; 返回 0 表示出错
28 done:
```

```
29     pop esi
30     pop edx
31     pop ecx
32     pop ebx
33     ret
34 dec2dw ENDP
```

5.2 关键代码分析

首先在.data 段预分配固定大小的内存缓冲区：20 字节输入缓冲区用于接收用户输入；9 字节输出缓冲区用于存储十六进制结果。

再调用 StdOut 函数向控制台输出输入提示字符串，将输入的字符序列存储到 inputBuf 缓冲区，自动在输入末尾添加 NULL 终止符。使用 strlen 获取实际输入长度，定位字符串末尾字符位置，检测并移除可能的回车符和换行符，从而确保缓冲区中只包含纯数字字符和终止符。

5.2.1 dec2dw 将字符串转十进制整数

1. 乘法操作版本的 dec2dw 过程关键代码语句如下：

```
1     imul eax, eax, 10
```

imul 是有符号整数乘法指令，这个语句其意为将当前累加结果乘 10。

2. 移位操作版本的 dec2dw 过程关键代码语句如下：

```
1     mov ebx, eax        ; 保存当前结果
2     shl eax, 1          ; EAX = EAX * 2
3     shl ebx, 3          ; EBX = EBX * 8
4     add eax, ebx        ; EAX = EAX + EBX = EAX * 10
```

依据数学原理 $x \times 10 = x \times 2 + x \times 8, (8 = 2^3)$ ，利用位操作指令 shl 把二进制数的所有位向左移动指定的位数，得到上述语句。

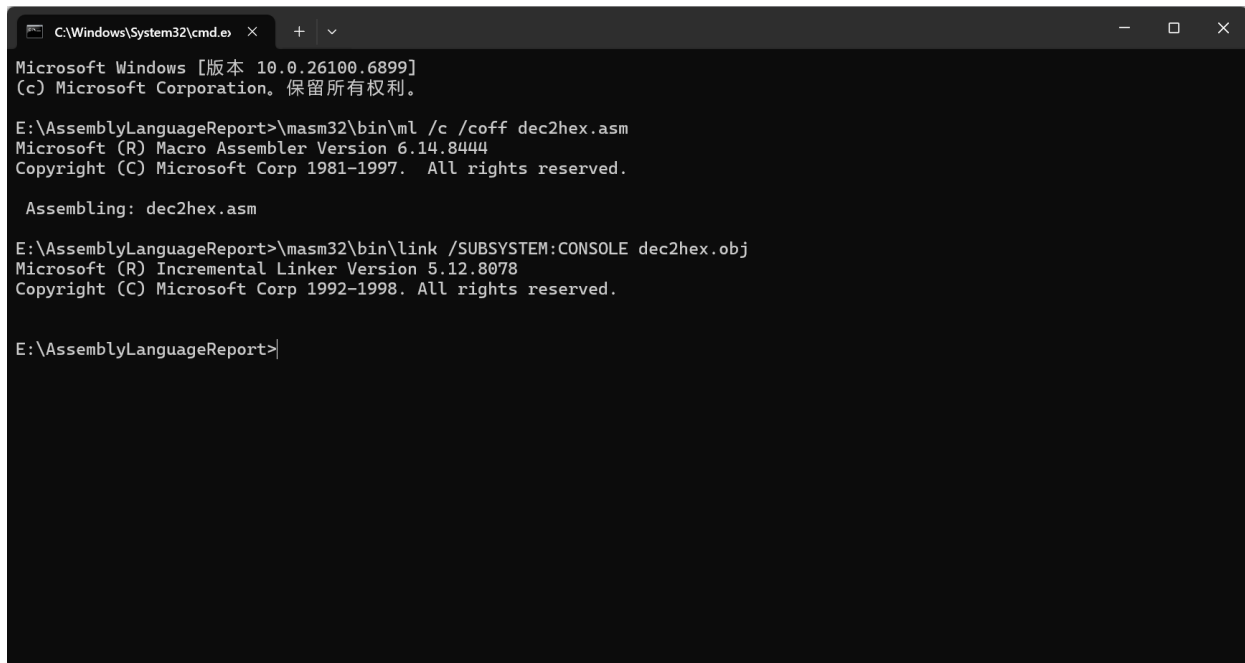
5.2.2 dw2hex 十进制整数转十六进制字符串

设置 8 次循环对应 32 位数据的 8 个十六进制数字，每次循环处理半个字节数据。之后使用 ROL 指令循环左移 4 位，将最高 4 位旋转到最低位置，通过位掩码 (0Fh) 提取最低 4 位数值。对于数值 0-9，直接加 '0' 得到对应 ASCII 数字字符；对于数值 10-15，加 '0' 后再加 7，得到 'A' 到 'F' 字符。将最后转换后的字符按顺序存入输出缓冲区，在 8 个字符后添加 NULL 终止符。

```
1 hex_loop:
2     rol ebx, 4          ; 左旋转 4 位，把最高 4 位移到最低位
3     mov dl, bl          ; 取 BL (低 8 位) 中的低 4 位
4     and dl, 0Fh         ; 屏蔽高 4 位，只保留低 4 位 (0~15)
5     cmp dl, 9           ; 判断是否为 0-9
6     jbe is_digit        ; 9 → 是数字，直接转 ASCII
```

7 `add dl, 7` ; >9 → 是 A-F, 需加 7

6 dec2hex.asm 编译和链接过程说明



```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26100.6899]
(c) Microsoft Corporation. 保留所有权利。

E:\AssemblyLanguageReport>\masm32\bin\ml /c /coff dec2hex.asm
Microsoft (R) Macro Assembler Version 6.14.8444
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.

Assembling: dec2hex.asm

E:\AssemblyLanguageReport>\masm32\bin\link /SUBSYSTEM:CONSOLE dec2hex.obj
Microsoft (R) Incremental Linker Version 5.12.8078
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.

E:\AssemblyLanguageReport>
```

图 6.1: 编译链接

- **`\masm32\bin\ml /c /coff dec2hex.asm`**

编译: 将汇编源文件 `dec2hex.asm` 翻译成目标文件 (`.obj`)

- `\masm32\bin\ml`: 调用 MASM 的汇编器程序 `ml.exe`
- `/c`: 表示只进行汇编, 不自动调用链接器, 即仅生成 `.obj` 文件
- `/coff`: 指定输出的目标文件格式为 COFF
- `dec2hex.asm`: 输入的汇编源代码文件名

- **`\masm32\bin\link /SUBSYSTEM:CONSOLE dec2hex.obj`**

链接: 将目标文件 `dec2hex.obj` 与必要的库文件合并生成最终的可执行文件 (`.exe`)

- `\masm32\bin\link`: 调用 Microsoft 的 32 位链接器 `link.exe`
- `/SUBSYSTEM:CONSOLE`: 指定程序运行环境为控制台
- `dec2hex.obj`: 要链接的目标文件

7 dec2hex.exe 的测试说明

这里测试了十进制数 100 和 0, 得到结果如下图:



```
E:\AssemblyLanguageReport\ >
Please input a decimal number (0 ~ 4294967295): 100
The hexadecimal number is : 00000064
```

图 7.2: 运行示例 1



```
E:\AssemblyLanguageReport\ >
Please input a decimal number (0 ~ 4294967295): 0
The hexadecimal number is : 00000000
```

图 7.3: 运行示例 2

测试案例通过。

8 实验结论及心得体会

8.1 实验结论

本程序成功实现了从十进制字符串到十六进制字符串的完整转换。

8.2 心得体会

本次的实验综合运用了字符串处理、数值转换、位运算等等知识。经过本次实验，我对汇编语言的理解有所加深。实验中修改错误的过程也锻炼了我的代码能力。是一次收获颇丰的实验。